

LED 102

Controlling multi-color LEDs with Python programs

In this activity, we'll learn how to use a tiny computer, called a microcontroller, to make an LED light up in a variety of colors.

Secondary skills we'll learn are: how to write simple Python programs and how to use Visual Studio Code to upload those programs to a microcontroller.

Materials Needed

1 ESP32-S3 SuperMini microcontroller. (Be sure to get one with a built-in WS2812b NeoPixel.)

1 Windows or Linux computer for the host PC.

1 USB-A to USB-C cable to connect computer to microcontroller.

We'll also need the ability to install programs (Visual Studio Code and extensions) and an internet connection to download MicroPython firmware and code.

1 Solderless breadboard (optional) to mount the SuperMini.

1 Bluetooth speaker (optional) for musical light show.

Facilitator Setup

The person running the activity should set up at least one ESP32-S3 SuperMini with MicroPython firmware and the color cycle Python program to use as a demo. Ideally, the facilitator will also have a host laptop set up with the software listed below in Participant setup.

In the interest of time and to make things easier for younger participants, the facilitator may want to have a number of ESP32-S3 SuperMinis already flashed with MicroPython ahead of time. This way, participants can concentrate on coding rather than firmware download and setup.

Participant Computer Setup

If participants are bringing their own laptop to use as a host PC, they can save time by installing the programs needed for coding with MicroPython on the ESP32 family of microcontrollers.

Visual Studio Code

- <https://code.visualstudio.com/download>

Python

- <https://www.python.org/downloads/>

Python Modules

- <https://pypi.org/project/esptool/>
- <https://pypi.org/project/mpremote/>

Visual Studio Code Extensions

- <https://marketplace.visualstudio.com/items?itemName=ms-python.python>
- <https://marketplace.visualstudio.com/items?itemName=DavesCodeMusings.esptool>
- <https://marketplace.visualstudio.com/items?itemName=DavesCodeMusings.mpremove>

There are many required software components, making this a non-trivial exercise. It would be wise to have a few host machines set up ahead of time or set aside thirty minutes or so for a guided installation and troubleshooting session.

Show and Tell Activity

Participants gather while the facilitator applies power to the ESP32-S3 SuperMini that has been preloaded with the color cycling program. Using a USB charger rather than plugging into a laptop USB port demonstrates that the laptop plays no part in the LED color changes and the ESP32-S3 SuperMini is working independently.

The facilitator give a high-level explanation of the programmable Red/Green/Blue (RGB) LED, explaining that it is comprised of three LEDs and each one can be configured with over two-hundred different brightness levels. By mixing different red, green, and blue brightness levels, the LED can theoretically create over sixteen million colors.

The color cycle program displays only six of the possible color combination.

Hands-On Activity

Have participants organize into groups of two or three, sharing host laptops if needed. Each group will attach one of the ESP32-S3 microcontrollers to their host machine.

With the facilitator's guidance, participants will use Visual Studio Code to explore the ESP32-S3 microcontroller and the MicroPython programs loaded on it.

Participants can explore by running the *SuperMini-MicroPython-NeoPixel-Color-Cycle.py* program at: <https://github.com/DavesCodeMusings/MakerSpace/tree/main/LEDs>

Intermediate Activity

Participants can modify the values in the color cycle program's list of colors in order to add, remove, or change the sequence shown by the LED. Brightness of the LED can also be changed by using values up to 255.

Advanced Activity

Participants can save their customized color cycle program to a file called *main.py* and upload it to the ESP32-S3 SuperMini. This will make the program start each time power is applied to the microcontroller, regardless of a host machine being attached or not.

Expert Activity

Participants can use the MicroPython interactive prompt (REPL) to manually step through Python statements that control the LED and see the effect as each line of code as it is executed. This is covered in the *SuperMini-MicroPython-NeoPixel.md* document, also at:

<https://github.com/DavesCodeMusings/MakerSpace/tree/main/LEDs>

Mad Science Activity

Photo-sensitive warning!

Participants can create a musical light show by replacing the `sleep()` function with `sleep_ms()` and specifying shorter transition delays. The `sleep_ms()` function takes milliseconds as a parameter, so 1 second is 1000 milliseconds, half a second is 500 milliseconds, and so on. Participants can use math or experimentation to arrive at a delay time that will change the LED color in time with the beat.

Helpful Formulas for Syncing Beats

Dividing BPM by 60 seconds/minute gives beats per second.

$$\text{bps} = \text{BPM} / 60$$

Dividing again by 1000 gives beats per millisecond.

$$\text{bpms} = \text{bps} / 1000$$

Period is the time between beats (expressed in milliseconds.)

$$\text{period} = 1 / \text{bpms}$$

Setting the delay at or near the period will help align the transitions to the beat.

For example, a progressive trance track is found to be running at 128 beats per minute. Beats per second is $128 / 60$ or 2.133 (repeating.) Beats per millisecond is $2.133 / 1000$ or 0.002133. Period is $1 / 0.002133$ or 468.75. A sleep value of 468 or 469 milliseconds should get close to LED transitions on the beat.

There are online tools that can help, like TapBPM at: <https://www.beatsperminuteonline.com/>